# LAB NO. 07

## ABSTRACTION

## (ABSTRACT CLASS WITH PURE VIRTUAL FUNCTIONS)

**Lab Objectives**

Following are the lab objectives:

1. Understand the concept of abstraction in Object-Oriented Programming.
2. Learn how to define abstract classes and pure virtual functions in C++.
3. Implement abstraction to hide implementation details and expose only essential features.
4. Apply abstraction to design flexible and extensible program structures.

| **Student ID** | **9864** | **Student Name** | **Ali Raza** |
|---|---|---|---|
| | **Marks** | **Obtained Marks** | **Comments** |
| **Task 1** | **20** | | |
| **Task 2** | **20** | | |
| **Task 3** | **20** | | |
| **Total Marks** | **60** | | |

**Lab Instructor**

# Pure Virtual Functions and Abstract Classes in C++

## What is an Abstract Class in C++?

An **abstract class** is a class that cannot be used to create objects directly. It is meant to be a **base class** that provides a common interface but **doesn't define full behavior**.
A class becomes abstract if it has at least one pure virtual function.

A class that contains a pure virtual function is known as an abstract class.

We cannot create objects of an abstract class. However, we can derive classes from them, and use their data members and member functions (except pure virtual functions).

## What is a Pure Virtual Function?

A **pure virtual function** is a function declared in a base class that **must be overridden** in any derived class.
It is written like this:

```
virtual void draw() = 0;  // Pure virtual function
```

- The `= 0` means **"no default implementation"**.
- It forces derived classes to **implement the function**.

### Example of Abstract Class and Pure Virtual Function

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0; // Pure virtual function
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle\n";
    }
};

int main() {
    // Shape s;       Error: Cannot create object of abstract class
    Shape* s = new Circle();  // Pointer to abstract class
    s->draw();                // Output: Drawing Circle
    delete s;
}
```

**Let's take an example,**

Suppose, we have derived Triangle, Square and Circle classes from the Shape class, and we want to calculate the area of all these shapes.

In this case, we can create a pure virtual function named calculateArea() in the Shape. Since it's a pure virtual function, all derived classes Triangle, Square and Circle must include the calculateArea() function with implementation.

A pure virtual function doesn't have the function body and it must end with = 0. For example:

```cpp
class Shape {
    public:

      // creating a pure virtual function
      virtual void calculateArea() = 0;
};
```

> **Note:** The `= 0` syntax doesn't mean we are assigning 0 to the function. It's just the way we define pure virtual functions.

In the above example, the class Shape is an abstract class.

## Example: C++ Abstract Class and Pure Virtual Function

```
// C++ program to calculate the area of a square and a circle

#include <iostream>
using namespace std;

// Abstract class
class Shape {
   protected:
    float dimension;

   public:
    void getDimension() {
        cin >> dimension;
    }

    // pure virtual Function
    virtual float calculateArea() = 0;
};

// Derived class
class Square : public Shape {
   public:
    float calculateArea() {
        return dimension * dimension;
    }
};

// Derived class
class Circle : public Shape {
   public:
    float calculateArea() {
        return 3.14 * dimension * dimension;
    }
};

int main() {
    Square square;
    Circle circle;

    cout << "Enter the length of the square: ";
```

```
    square.getDimension();
    cout << "Area of square: " << square.calculateArea() << endl;

    cout << "\nEnter radius of the circle: ";
    circle.getDimension();
    cout << "Area of circle: " << circle.calculateArea() << endl;

    return 0;
}
```

**Output**

```
Enter the length of the square: 4
Area of square: 16

Enter radius of the circle: 5
Area of circle: 78.5
```

In this program, `virtual float calculateArea() = 0;` inside the `Shape` class is a pure virtual function.

That's why we must provide the implementation of `calculateArea()` in both of our derived classes, or else we will get an error.

# LAB TASKS

## Task 1 (20 mark)

Scenario: A university wants to manage different types of staff members (e.g., **Teacher**, **Clerk**, and **Librarian**).

- Create an abstract class **Staff** with a pure virtual function calculateSalary().
- Derive classes **Teacher**, **Clerk**, and **Librarian** that implement the function differently (e.g., teacher based on lectures, clerk on hours worked, librarian on number of books managed).
- Demonstrate how abstraction allows defining a common interface but different implementations.

Program:

```
#include<iostream>

using namespace std;


class staff {

public:

	virtual void calculatesalary() = 0;

};


class teacher : public staff {

public:

	void calculatesalary() {

		int lecture = 20;

		int rate = 50;
```

```
            cout << "Teacher Salary is : " << lecture * rate << endl;
        }
};

class clerk : public staff {
public:
        void calculatesalary() {
            int hours = 10;
            int rate = 20;
            cout << "Salary of clerk is : " << hours * rate << endl;
        }
};

class librarian : public staff {
public:
        void calculatesalary() {
            int books = 5;
            int rate = 50;
            cout << "Salary of librarian is : " << books * rate << endl;
        }
};
```

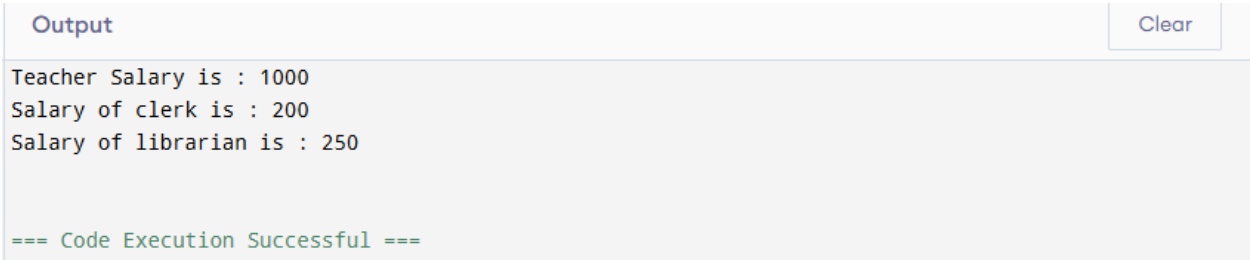
```
int main() {
        staff* s1 = new teacher();
        staff* s2 = new clerk();
        staff* s3 = new librarian();

        s1->calculatesalary();
        s2->calculatesalary();
        s3->calculatesalary();

        delete s1;
        delete s2;
        delete s3;

        return 0;
}
```

Output:




**Task 2** **(20 mark)**

***Scenario:*** A transport company has different vehicles (**Car**, **Bike**, and **Bus**).

- Create an abstract class **Vehicle** with a pure virtual function getFuelEfficiency().
- Each derived class should provide its own logic (e.g., Car = km/l, Bike = km/l, Bus = liters/100km).
- Write a program where the user enters details and the program outputs the fuel efficiency of each vehicle.

## Program:

```
#include <iostream>

using namespace std;


class Vehicle {

public:

   virtual void getFuelEfficiency() = 0;

};


class Car : public Vehicle {

private:

   double distance = 300.0; // km

   double fuel = 20.0;      // liters

public:

   void getFuelEfficiency() override {

      double efficiency = distance / fuel;

      cout << "Car Fuel Efficiency = " << efficiency << " km/l" << endl;

   }
```

```
};


class Bike : public Vehicle {
private:
    double distance = 150.0; // km
    double fuel = 5.0;       // liters
public:
    void getFuelEfficiency() override {
        double efficiency = distance / fuel;
        cout << "Bike Fuel Efficiency = " << efficiency << " km/l" << endl;
    }
};


class Bus : public Vehicle {
private:
    double distance = 400.0; // km
    double fuel = 80.0;      // liters
public:
    void getFuelEfficiency() override {
        double efficiency = (fuel / distance) * 100;
```

```
        cout << "Bus Fuel Efficiency = " << efficiency << " liters/100km" << endl;
    }
};

int main() {
    Vehicle* v;

    Car c;
    Bike b;
    Bus bs;

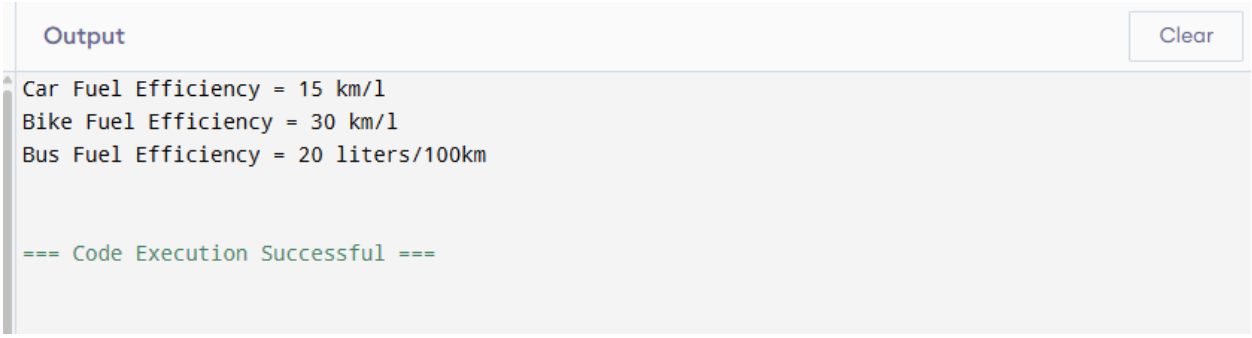
    v = &c;
    v->getFuelEfficiency();

    v = &b;
    v->getFuelEfficiency();

    v = &bs;
    v->getFuelEfficiency();

    return 0;
}
```

## Output:




**Task 3** **(20 mark)**

*Scenario:* An online shopping system has different types of payment methods (**CreditCard**, **PayPal**, **CashOnDelivery**).

- Create an abstract class **Payment** with a pure virtual function processPayment(float amount).
- Each derived class should override the function to display how the payment is processed.
- In main(), simulate a shopping cart checkout using different payment methods to show abstraction in action.

## Program:

```
#include <iostream>

using namespace std;


class Payment {

public:

   virtual void processpayment(float amount) = 0;

};

class CreditCard : public Payment {
```

```
public:
    void processpayment(float amount) {
        cout << "credit card payment done of " << amount << " rs" << endl;
    }
};
class PayPal : public Payment {
public:
    void processpayment(float amount) {
        cout << "paypal payment done of " << amount << " rs" << endl;
    }
};

class CashOnDelivery : public Payment {
public:
    void processpayment(float amount) {
            cout << "cash on delivery payment of " << amount << " rs when parcel arrives" << endl;
    }
};

int main() {
    CreditCard c;
```

```
    PayPal p;
    CashOnDelivery cod;
    c.processpayment(2500);
    p.processpayment(1800);
    cod.processpayment(3200);

    return 0;
}
```

## Output:

